

ESWIN

ESSDK系统控制开发者手册

Rev 1.0

2026-04-17

声明

知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

文档版本	日期	说明
v1.0	2026-04-17	新增 Python API 章节，与 C API 合并整理
v0.8	2024-12-20	增加了在配置文件中修改日志级别的说明
v0.7	2024-08-28	修改测试团队 review 意见
v0.6	2024-07-08	更新部分函数描述
v0.5	2024-06-28	修改测试团队 review 意见
v0.4	2024-06-04	初始版本

目录

声明	1
知识产权声明	1
责任限制	1
修订记录	2
1. 概述	5
1.1. 基本介绍	5
1.2. 基本概念	5
1.2.1. 系统管理	5
1.2.2. 版本管理	5
1.2.3. 内存管理	5
1.2.4. 日志管理	5
1.2.5. 时间管理	5
1.2.6. 绑定管理 (Binding)	5
2. C API 接口说明	7
2.1. 系统管理	7
2.1.1. ES_SYS_Init	7
2.1.2. ES_SYS_Exit	7
2.2. 版本管理	7
2.2.1. ES_SYS_GetVersion	7
2.3. 日志管理	8
2.3.1. ES_SYS_LogPrint	8
2.4. 时间管理	9
2.4.1. ES_SYS_GetCurCnt	9
2.5. 绑定管理	9
2.5.1. ES_SYS_Bind	9
2.5.2. ES_SYS_UnBind	10
2.5.3. ES_SYS_GetBindbyDest	10
2.5.4. ES_SYS_GetBindbySrc	11
2.6. 数据类型	11
2.6.1. SYS_VERSION_S	11
2.6.2. CHN_S	12
2.6.3. BIND_DEST_S	12
2.6.4. MOD_ID_E	13
2.6.5. ES_LOG_LEVEL_E	14
3. Python API 接口说明	15
3.1. 系统管理	15
3.1.1. init()	15
3.1.2. exit()	15
3.2. 版本管理	16
3.2.1. get_version()	16
3.3. 日志管理	16
3.3.1. log_print()	16
3.4. 时间管理	17
3.4.1. get_cur_cnt()	17
3.5. 绑定管理	18
3.5.1. bind()	18
3.5.2. unbind()	19
3.5.3. get_bind_by_dest()	19
3.5.4. get_bind_by_src()	19
3.6. 数据类型	20
3.6.1. SYS_VERSION_S	20

3.6.2. CHN_S	20
3.6.3. BIND_DEST_S	21
3.6.4. MOD_ID_E	21
3.6.5. ES_LOG_LEVEL_E	23
4. 错误码	24

1. 概述

1.1. 基本介绍

SYS (System Control, 系统控制) 模块是 ESSDK 的基础核心组件, 主要为多媒体系统及其他子系统提供公共服务。它负责管理整个 SDK 的生命周期、资源协调以及模块间的数据路由。SYS 模块的功能涵盖了系统管理、版本管理、内存管理、日志管理、时间管理以及绑定管理等核心功能。

为了满足不同开发场景的需求, SYS 模块提供了 **C 语言 API** 和 **Python API** 两套接口:

- **C API**: 作为底层核心接口, 提供了最高的执行效率和直接的控制能力, 适用于对性能和实时性有严格要求的嵌入式应用场景。相关库文件为 `libes_sys.so`, 头文件为 `es_sys.h` 和 `es_comm_sys.h`。
- **Python API**: 提供了更加简洁、符合 Python 编程习惯的高级封装。它将底层的错误码转换为异常机制, 简化了资源管理流程, 适用于快速原型开发、算法验证及高层应用逻辑的开发。Python 模块可通过 `from es_py import sys` 导入。

1.2. 基本概念

1.2.1. 系统管理

负责 SDK 的初始化及反初始化操作。通过 `ES_SYS_Init` (C) 或 `sys.init()` (Python) 进行初始化, 为后续其他子模块的启动做准备; 通过 `ES_SYS_Exit` 或 `sys.exit()` 进行反初始化, 释放系统资源。

1.2.2. 版本管理

提供获取 ESSDK 版本信息的功能, 通过 `ES_SYS_GetVersion` 或 `sys.get_version()` 接口实现, 可用于兼容性校验或日志输出。

1.2.3. 内存管理

提供系统级的内存协调功能, 具体的内存分配与管理机制请参见《[Memory 用户手册](#)》。

1.2.4. 日志管理

提供系统中记录、存储和日志信息管理的功能。通过 `ES_SYS_LogPrint` 或 `sys.log_print()` 接口, 可将日志输出到 `syslog` 或者控制台, 并支持分级 (Emergency 至 Debug) 和基于模块 ID 的过滤。

1.2.5. 时间管理

提供高精度计时功能。`ES_SYS_GetCurCnt` 或 `sys.get_cur_cnt()` 接口返回一个 24 MHz 频率的计数值 (精度约为 42 ns), 主要用于性能分析和时间戳计算。

1.2.6. 绑定管理 (Binding)

绑定是 SYS 模块的核心机制之一, 通过调用绑定接口, 可以使数据源 (Sender) 和数据消费者 (Receiver) 之间建立自动化的数据流转关系。绑定后, 数据源生成的数据将自动发送给接收者, 无需用户手动干预。

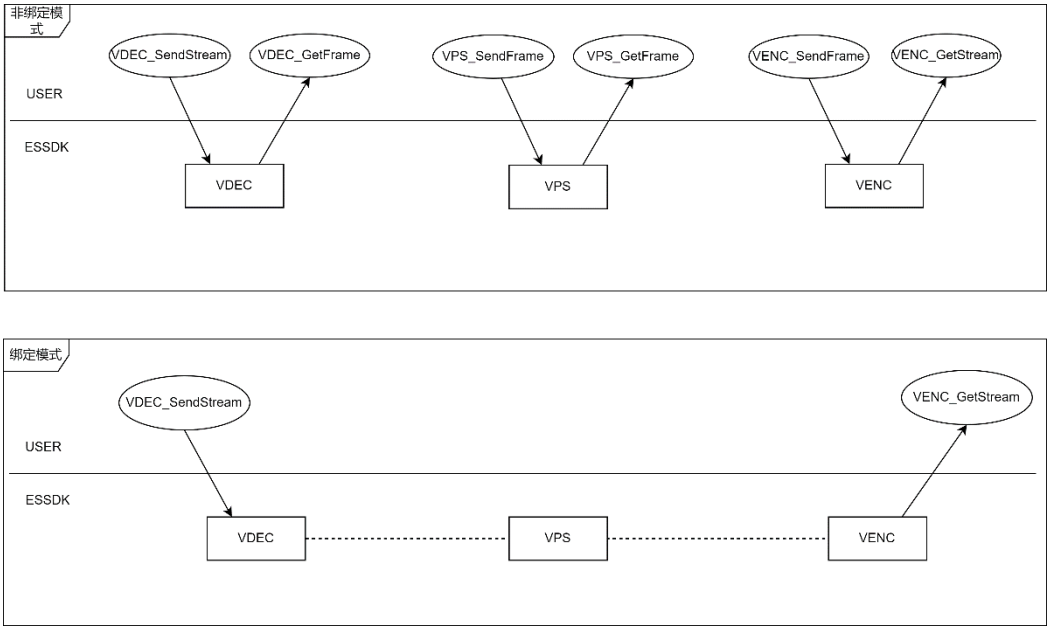


图 1: 绑定和非绑定区别

绑定关系规则:

并非所有模块之间都可以随意绑定，支持绑定的模块拓扑如下表所示：

数据发送端 (Source)	数据接收端 (Destination)
VPS	VO
	VENC
VDEC	VPS
	VENC
	VO
VO	VENC
	VPS
AI	AENC
	AO
ADEC	AO

模块角色说明:

- **源端 (Source Only):** VDEC、AI、ADEC 只能作为绑定源端，发送数据。
- **目的端 (Destination Only):** VENC、AENC 只能作为绑定目的端，接收数据。
- **双向:** VO、VPS 既可作为绑定源又可作为绑定目的端，支持发送和接收数据。

拓扑约束:

- **一对多:** 一个发送端可以绑定到多个不同的接收端。
- **一对一:** 一个接收端只能有一个发送端。

2. C API 接口说明

SYS 的 C 语言接口是系统的基础 API，库文件为 `libes_sys.so`，头文件为 `es_sys.h` 和 `es_comm_sys.h`。本章节将详细介绍系统管理、版本管理、日志管理、时间管理及绑定管理等功能的 C 接口用法。

2.1. 系统管理

系统管理提供 SYS 资源的初始化和反初始化功能，是所有其他模块运行的前提。

2.1.1. ES_SYS_Init

【函数体】

```
ES_S32 ES_SYS_Init(ES_VOID);
```

【描述】

初始化 SYS 模块资源。在所有其他 ESSDK 模块调用之前，**必须**首先调用此接口。

【参数】

无

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.1.2. ES_SYS_Exit

【函数体】

```
ES_S32 ES_SYS_Exit(ES_VOID);
```

【描述】

反初始化 SYS 模块，释放已分配的系统资源。程序退出前必须调用。

【参数】

无

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2. 版本管理

2.2.1. ES_SYS_GetVersion

【函数体】


```
ES_S32 ES_SYS_GetVersion(SYS_VERSION_S *pVersion);
```

【描述】

获取 ESSDK 的版本信息字符串。

【参数】

参数名称	描述	输入/输出
pVersion	指向 SYS_VERSION_S 结构体的指针	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.3. 日志管理

2.3.1. ES_SYS_LogPrint

【函数体】

```
ES_S32 ES_SYS_LogPrint(  
    ES_LOG_LEVEL_E level,  
    const ES_CHAR *pTag,  
    MOD_ID_E id,  
    ES_CHAR const *pFormat,  
    ...  
);
```

【描述】

格式化输出日志到 syslog 或控制台。可为 Log 输出源指定 tag 和 id，便于高效地进行模块匹配和过滤。系统使用 logrotate 对日志文件大小进行管理，定期备份。

【参数】

参数名称	描述	输入/输出
level	日志级别	输入
pTag	Log 输出模块的字符串标记 (Tag)	输入
id	模块 ID，取值范围参考 MOD_ID_E 定义	输入
pFormat	格式化字符串 (类似 printf)	输入

【返回值】

返回值	描述
0	成功

返回值	描述
非 0	失败，参见 错误码

提示

- 默认将日志输出到 `syslog` 文件中。
- 如需输出到控制台，需设置环境变量 `export ES_SYSLOG=n` 并重启相关程序。
- 如需修改日志级别，请编辑 `/etc/es_syslog.conf`，修改后重启应用程序生效。

2.4. 时间管理

2.4.1. ES_SYS_GetCurCnt

【函数体】

```
ES_S32 ES_SYS_GetCurCnt(ES_U32 *pValue);
```

【描述】

获取当前系统计数器的计数值。该计数器频率为 24 MHz，精度约为 42 ns。由于使用 32 位计数器，大约每隔 180 秒会发生回绕。

【参数】

参数名称	描述	输入/输出
pValue	用于存放读取结果的指针	输出

【返回值】

返回值	描述
0	成功
非 0	计数器未初始化

2.5. 绑定管理

绑定管理用于在数据源（Sender）和数据消费者（Receiver）之间建立或解除关联关系。

2.5.1. ES_SYS_Bind

【函数体】

```
ES_S32 ES_SYS_Bind(const CHN_S *pSrcChn, const CHN_S *pDestChn);
```

【描述】

在数据发送者和接收者之间建立绑定关系。绑定后数据将自动流转。

【参数】

参数名称	描述	输入/输出
pSrcChn	发送端通道结构体指针	输入
pDestChn	接收端通道结构体指针	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.5.2. ES_SYS_UnBind

【函数体】

```
ES_S32 ES_SYS_UnBind(const CHN_S *pSrcChn, const CHN_S *pDestChn);
```

【描述】

解除指定发送端与接收端之间的绑定关系。

【参数】

参数名称	描述	输入/输出
pSrcChn	发送端通道结构体指针	输入
pDestChn	接收端通道结构体指针	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.5.3. ES_SYS_GetBindbyDest

【函数体】

```
ES_S32 ES_SYS_GetBindbyDest(const CHN_S *pDestChn, CHN_S *pSrcChn);
```

【描述】

通过绑定接收端查询与其关联的发送端信息。

【参数】

参数名称	描述	输入/输出
pDestChn	接收端结构体指针	输入
pSrcChn	发送端结构体指针	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

【注意】一个接收端仅有一个发送端。

2.5.4. ES_SYS_GetBindbySrc

【函数体】

```
ES_S32 ES_SYS_GetBindbySrc(const CHN_S *pSrcChn, BIND_DEST_S *pBindDest);
```

【描述】

通过绑定发送端查询与其关联的所有接收端信息。

【参数】

参数名称	描述	输入/输出
pSrcChn	发送端结构体指针	输入
pBindDest	接收端结构体指针，包含所有接收者信息	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

同一个发送端可以有多个不同的接收端。

2.6. 数据类型

2.6.1. SYS_VERSION_S

【说明】

定义版本信息结构体，用于存储 ESSDK 的版本字符串。

【定义】

```
#define VERSION_NAME_MAXLEN 64

typedef struct esSYS_VERSION_S {
    ES_CHAR version[VERSION_NAME_MAXLEN];
} SYS_VERSION_S;
```

【成员】

成员名称	描述
version	版本信息字符串，以字符 '\0' 结束

2.6.2. CHN_S

【说明】

定义绑定发送端与接收端的通道信息结构体。

【定义】

```
typedef struct esCHN_S {
    MOD_ID_E modId; // 模块号
    ES_S32 nId;     // Die Id
    ES_S32 devId;   // 设备或组编号
    ES_S32 chnId;   // 通道号
} CHN_S;
```

【成员】

成员名称	描述
modId	模块 ID，具体参见 MOD_ID_E 定义
nId	Die Id
devId	设备或组编号，不同的模块有不同的意义
chnId	通道号，取值范围请见各个模块的 chn 取值范围

【取值范围】

devId 和 chnId 的具体取值范围如下表所示：

模块名称	devId 范围	chnId 范围
VENC	未使用	[0, ES_VENC_MAX_CHN_NUM)
VDEC	[0, ES_VDEC_MAX_GRP_NUM)	[0, ES_VDEC_OUT_CHN_NUM)
VPS	[0, ES_VPS_MAX_GRP_NUM)	[0, ES_VPS_MAX_CHN_NUM)
VO	[0, ES_VO_MAX_DEV_NUM)	[0, ES_VO_MAX_CHN_NUM)

2.6.3. BIND_DEST_S

【说明】

定义获取所有绑定接收端的结构体。

【定义】

```
#define BIND_DEST_MAXNUM 128

typedef struct esBIND_DEST_S {
    ES_U32 num;
    CHN_S chn[BIND_DEST_MAXNUM];
}
```

```
} BIND_DEST_S;
```

【成员】

成员名称	描述
num	实际的绑定接收端个数，最大值为 BIND_DEST_MAXNUM
chn	存放接收端信息的数组，具体参见 CHN_S

2.6.4. MOD_ID_E

【说明】

定义模块 ID 枚举。

【定义】

```
typedef enum esMOD_ID_E {
    ES_ID_VB = 1,
    ES_ID_SYS = 2,
    ES_ID_VDEC = 3,
    ES_ID_VPS = 4,
    ES_ID_VENC = 5,
    ES_ID_VO = 6,
    ES_ID_VI = 7,
    ES_ID_AIO = 8,
    ES_ID_AI = 9,
    ES_ID_AO = 10,
    ES_ID_AENC = 11,
    ES_ID_ADEC = 12,
    ES_ID_USER = 13,
    ES_ID_GDC = 14,
    ES_ID_NPU = 15,
    ES_ID_DSP = 16,
    ES_ID_BMS = 17,
    ES_ID_NUMA = 18,
    ES_ID_CIPHER = 19,
    ES_ID_AK = 20,
    ES_ID_MEMCP = 21,
    ES_ID_BUTT
} MOD_ID_E;
```

【成员】

成员名称	描述
ES_ID_VB	内存缓存模块 ID
ES_ID_SYS	系统控制模块 ID
ES_ID_VDEC	视频解码模块 ID
ES_ID_VPS	视频处理模块 ID
ES_ID_VENC	视频编码模块 ID
ES_ID_VO	视频输出模块 ID
ES_ID_AI	音频输入模块 ID
ES_ID_AO	音频输出模块 ID

成员名称	描述
ES_ID_AENC	音频编码模块 ID
ES_ID_ADEC	音频解码模块 ID
ES_ID_NPU	NPU 模块 ID
ES_ID_DSP	DSP 模块 ID
ES_ID_BMS	绑定管理模块 ID，仅内部使用
ES_ID_NUMA	Numa 模块 ID
ES_ID_CIPHER	Cipher 模块 ID
ES_ID_AK	AK 模块 ID
ES_ID_MEMCP	DMA 内存拷贝模块 ID

2.6.5. ES_LOG_LEVEL_E

【说明】

定义日志级别枚举。

【定义】

```
typedef enum {
    ES_LOG_MIN = -1,
    ES_LOG_EMERGENCY = 0,
    ES_LOG_ALERT = 1,
    ES_LOG_CRITICAL = 2,
    ES_LOG_ERROR = 3,
    ES_LOG_WARN = 4,
    ES_LOG_NOTICE = 5,
    ES_LOG_INFO = 6,
    ES_LOG_DEBUG = 7,
    ES_LOG_MAX
} ES_LOG_LEVEL_E;
```

【成员】

成员名称	值	描述
ES_LOG_MIN	-1	最低日志等级，可能用于无效或未定义的状态
ES_LOG_EMERGENCY	0	紧急：系统不可用，可能导致系统崩溃的严重问题，需要立即处理
ES_LOG_ALERT	1	警报：必须立即采取行动的严重问题，例如系统关键组件的故障
ES_LOG_CRITICAL	2	严重：关键条件，可能导致服务中断或系统瘫痪的问题
ES_LOG_ERROR	3	错误：一般错误信息，可能导致某些功能失败或不稳定
ES_LOG_WARN	4	警告：潜在的问题，但不会立即导致系统或服务的中断
ES_LOG_NOTICE	5	通知：重要的正常运行信息，通常用作系统事件的提示或关注点
ES_LOG_INFO	6	信息：一般的运行信息，通常用于记录系统状态和操作细节
ES_LOG_DEBUG	7	调试：用于开发或诊断时的详细调试信息
ES_LOG_MAX	8	最大日志等级标记，不用于实际记录日志

3. Python API 接口说明

Python 接口封装在 `es_py.sys` 模块中，提供了与 C 接口一一对应的功能，同时符合 Python 的编程习惯（如使用异常代替错误码）。

与系统接口相关的结构体、枚举和常量主要定义在 `es_py.common` 模块中。

导入方式：

```
from es_py import sys
```

3.1. 系统管理

本节提供系统初始化和反初始化 API。

3.1.1. init()

【函数原型】

```
def init() -> int
```

【功能描述】

初始化系统控制模块。必须在操作其他模块之前调用该函数。

【参数】

无

【返回值】

成功返回 0。

【异常】

如果初始化失败，会抛出 `ESSysError` 异常并包含错误码。

【示例】

```
try:
    ret = sys.init()
    print("系统初始化成功")
except Exception as e:
    print(f"系统初始化失败: {e}")
```

3.1.2. exit()

【函数原型】

```
def exit() -> int
```

【功能描述】

反初始化系统控制模块，释放已分配的资源。程序退出前必须调用。

【参数】

无

【返回值】

成功返回 0。

【异常】

如果反初始化失败，会抛出 `ESSysError` 异常并包含错误码。

【示例】

```
# ... 使用系统功能 ...  
sys.exit()
```

3.2. 版本管理

本节提供获取 ESSDK 版本信息的 API。

3.2.1. get_version()

【函数原型】

```
def get_version() -> str
```

【功能描述】

获取 ESSDK 版本信息字符串。

【参数】

无

【返回值】

返回版本信息字符串。

【异常】

如果获取失败，会抛出 `ESSysError` 异常。

【示例】

```
version = sys.get_version()  
print(f"ESSDK 版本: {version}")
```

3.3. 日志管理

本节提供日志输出 API。

3.3.1. log_print()

【函数原型】

```
def log_print(level: int, tag: str, id: MOD_ID_E, fmt: str) -> None
```

【功能描述】

日志输出函数，可为 Log 输出源指定 tag 和 id。系统使用 logrotate 对日志文件大小进行管理。

【参数】

参数名称	类型	描述
level	int	日志级别 (0~7)
tag	str	日志输出模块的 tag 标记
id	MOD_ID_E	日志输出的模块 ID
fmt	str	日志输出的消息内容

【返回值】

无

【示例】

```
from es_py import sys, common

# 输出 INFO 级别日志
sys.log_print(6, "MyApp", common.MOD_ID_E.ES_ID_SYS, "应用程序启动成功")

# 输出 ERROR 级别日志
sys.log_print(3, "MyApp", common.MOD_ID_E.ES_ID_SYS, "发生错误, 错误码: 0x80004001")
```

提示

- 默认将日志输出到 syslog 文件中。
- 如需输出到控制台，需设置环境变量 export ES_SYSLOG=n 并重启相关程序。
- 如需修改日志级别，请编辑 /etc/es_syslog.conf，修改后重启应用程序生效。

3.4. 时间管理

本节提供获取当前计数值的 API。

3.4.1. get_cur_cnt()

【函数原型】

```
def get_cur_cnt() -> int
```

【功能描述】

获取当前计数器的计数值。计数器频率为 24MHz，精度为 42ns。使用 32 位计数器，大约每隔 180 秒会发生回绕。

【参数】

无

【返回值】

返回当前计数器的计数值。

【异常】

如果计数器未初始化，会抛出 ESSysError 异常。

【示例】

```
# 获取当前计数值
cnt1 = sys.get_cur_cnt()
# ... 执行某些操作 ...
cnt2 = sys.get_cur_cnt()
elapsed_ns = (cnt2 - cnt1) * 42 # 转换为纳秒
print(f"操作耗时: {elapsed_ns} ns")
```

3.5. 绑定管理

本节提供数据通道绑定管理 API。

3.5.1. bind()

【函数原型】

```
def bind(src: CHN_S, dst: CHN_S) -> int
```

【功能描述】

在数据发送者和接收者之间建立绑定关系。

【参数】

参数名称	类型	描述
src	CHN_S	发送端通道信息
dst	CHN_S	接收端通道信息

【返回值】

成功返回 0。

【异常】

如果绑定失败，会抛出 `ESSysError` 异常并包含错误码。

【示例】

```
from es_py import sys, common

# 创建源通道和目标通道
src_chn = common.CHN_S()
src_chn.modId = common.MOD_ID_E.ES_ID_VDEC
src_chn.nId = 0
src_chn.devId = 0
src_chn.chnId = 0

dst_chn = common.CHN_S()
dst_chn.modId = common.MOD_ID_E.ES_ID_VPS
dst_chn.nId = 0
dst_chn.devId = 0
dst_chn.chnId = 0

# 绑定通道
sys.bind(src_chn, dst_chn)
```

3.5.2. unbind()

【函数原型】

```
def unbind(src: CHN_S, dst: CHN_S) -> int
```

【功能描述】

解除绑定关系。

【参数】

参数名称	类型	描述
src	CHN_S	发送端通道信息
dst	CHN_S	接收端通道信息

【返回值】

成功返回 0。

【异常】

如果解绑失败，会抛出 `ESSysError` 异常并包含错误码。

3.5.3. get_bind_by_dest()

【函数原型】

```
def get_bind_by_dest(dst: CHN_S) -> CHN_S
```

【功能描述】

通过绑定接收端查询与其关联的发送端信息。

【参数】

参数名称	类型	描述
dst	CHN_S	接收端通道信息

【返回值】

返回发送端通道信息 (CHN_S 对象)。

【异常】

如果查询失败，会抛出 `ESSysError` 异常。

提示

一个接收端仅有一个发送端。

3.5.4. get_bind_by_src()

【函数原型】

```
def get_bind_by_src(src: CHN_S) -> BIND_DEST_S
```

【功能描述】

通过绑定发送端查询与其关联的所有接收端信息。

【参数】

参数名称	类型	描述
src	CHN_S	发送端通道信息

【返回值】

返回 BIND_DEST_S 对象，包含所有接收者信息。

【异常】

如果查询失败，会抛出 `ESSysError` 异常。

提示

同一个发送端可以有多个不同的接收端。

3.6. 数据类型

Python 接口通过类（class）来模拟 C 的结构体，提供更面向对象的操作方式。以下类型主要由 `es_py.common` 模块导出。

3.6.1. SYS_VERSION_S

【定义】

```
class SYS_VERSION_S:
    def __init__(self):
        self.version: str = "" # 版本信息字符串
```

【成员】

成员名称	类型	描述
version	str	版本信息字符串

【相关接口】`get_version()`

3.6.2. CHN_S

【定义】

```
class CHN_S:
    def __init__(self):
        self.modId: MOD_ID_E = None # 模块号
        self.nId: int = 0           # Die ID
        self.devId: int = 0         # 设备或组编号
```

```
self.chnId: int = 0      # 通道号
```

【成员】

成员名称	类型	描述
modId	MOD_ID_E	模块 ID，具体参见 MOD_ID_E 定义
nId	int	Die ID
devId	int	设备或组编号，不同的模块有不同的意义
chnId	int	通道号，取值范围请见各个模块的 chn 取值范围

【取值范围】

与 C 接口中的 CHN_S 保持一致，具体参考下表：

模块名称	devId 范围	chnId 范围
venc	未使用	[0, ES_VENC_MAX_CHN_NUM)
vdec	[0, ES_VDEC_MAX_GRP_NUM)	[0, ES_VDEC_OUT_CHN_NUM)
vps	[0, ES_VPS_MAX_GRP_NUM)	[0, ES_VPS_MAX_CHN_NUM)
vo	[0, ES_VO_MAX_DEV_NUM)	[0, ES_VO_MAX_CHN_NUM)
ai	[0, ES_AI_MAX_DEV_NUM)	[0, ES_AI_MAX_CHN_NUM)
ao	[0, ES_AO_MAX_DEV_NUM)	[0, ES_AO_MAX_CHN_NUM)
aenc	未使用	[0, ES_AENC_MAX_CHN_NUM)
adec	[0, ES_ADEC_MAX_CHN_NUM)	未使用

3.6.3. BIND_DEST_S

【定义】

```
class BIND_DEST_S:
    def __init__(self):
        self.num: int = 0
        self.chn: FixedCArrayProxy[CHN_S] # 记录所有接收者的只读代理，最大 BIND_DEST_MAXNUM 个
```

【成员】

成员名称	类型	描述
num	int	实际的绑定接收端个数（最大 128 个）
chn	FixedCArrayProxy[CHN_S]	存放接收端信息的只读代理

3.6.4. MOD_ID_E

【说明】

Python 中使用 IntEnum 或特定类进行封装。

【定义示例】

```
from enum import IntEnum

class MOD_ID_E(IntEnum):
    ES_ID_VB = 1
    ES_ID_SYS = 2
    ES_ID_VDEC = 3
    ES_ID_VPS = 4
    ES_ID_VENC = 5
    ES_ID_VO = 6
    ES_ID_VI = 7
    ES_ID_AIO = 8
    ES_ID_AI = 9
    ES_ID_AO = 10
    ES_ID_AENC = 11
    ES_ID_ADEC = 12
    ES_ID_USER = 13
    ES_ID_GDC = 14
    ES_ID_NPU = 15
    ES_ID_DSP = 16
    ES_ID_BMS = 17
    ES_ID_NUMA = 18
    ES_ID_CIPHER = 19
    ES_ID_AK = 20
    ES_ID_MEMCP = 21
    ES_ID_BUTT = 22
```

【成员】

成员名称	描述
ES_ID_VB	内存缓存模块 ID
ES_ID_SYS	系统控制模块 ID
ES_ID_VDEC	视频解码模块 ID
ES_ID_VPS	视频处理模块 ID
ES_ID_VENC	视频编码模块 ID
ES_ID_VO	视频输出模块 ID
ES_ID_VI	视频输入模块 ID
ES_ID_AIO	音频输入输出模块 ID
ES_ID_AI	音频输入模块 ID
ES_ID_AO	音频输出模块 ID
ES_ID_AENC	音频编码模块 ID
ES_ID_ADEC	音频解码模块 ID
ES_ID_USER	用户模块 ID
ES_ID_GDC	GDC 模块 ID
ES_ID_NPU	NPU 模块 ID
ES_ID_DSP	DSP 模块 ID
ES_ID_BMS	绑定管理模块 ID，仅内部使用
ES_ID_NUMA	NUMA 模块 ID
ES_ID_CIPHER	Cipher 模块 ID
ES_ID_AK	AK 模块 ID

成员名称	描述
ES_ID_MEMCP	MEMCP 模块 ID
ES_ID_BUTT	枚举边界值

3.6.5. ES_LOG_LEVEL_E

【说明】

log_print() 的 level 参数底层会转换为 ES_LOG_LEVEL_E。

【定义】

```
from enum import IntEnum

class ES_LOG_LEVEL_E(IntEnum):
    ES_LOG_MIN = -1      # 最低日志等级
    ES_LOG_EMERGENCY = 0 # 紧急
    ES_LOG_ALERT = 1     # 警报
    ES_LOG_CRITICAL = 2  # 严重
    ES_LOG_ERROR = 3     # 错误
    ES_LOG_WARN = 4      # 警告
    ES_LOG_NOTICE = 5    # 通知
    ES_LOG_INFO = 6      # 信息
    ES_LOG_DEBUG = 7     # 调试
    ES_LOG_MAX = 8       # 最大日志等级
```

【成员】

成员名称	值	描述
ES_LOG_MIN	-1	最低日志等级，可能用于无效或未定义的状态
ES_LOG_EMERGENCY	0	紧急：系统不可用，可能导致系统崩溃的严重问题，需要立即处理
ES_LOG_ALERT	1	警报：必须立即采取行动的严重问题，例如系统关键组件的故障
ES_LOG_CRITICAL	2	严重：关键条件，可能导致服务中断或系统瘫痪的问题
ES_LOG_ERROR	3	错误：一般错误信息，可能导致某些功能失败或不稳定
ES_LOG_WARN	4	警告：潜在的问题，但不会立即导致系统或服务的中断
ES_LOG_NOTICE	5	通知：重要的正常运行信息，通常用作系统事件的提示或关注点
ES_LOG_INFO	6	信息：一般的运行信息，通常用于记录系统状态和操作细节
ES_LOG_DEBUG	7	调试：用于开发或诊断时的详细调试信息
ES_LOG_MAX	8	最大日志等级标记，不用于实际记录日志

4. 错误码

SYS 相关 API 错误码如下表所示。

错误码	宏定义	描述
0xA0028006	ES_ERR_SYS_NULL_PTR	函数参数中有空指针
0xA0028010	ES_ERR_SYS_NOTREADY	系统未初始化
0xA0028009	ES_ERR_SYS_NOT_PERM	操作不允许
0xA002800C	ES_ERR_SYS_NOMEM	内存不足
0xA0028003	ES_ERR_SYS_ILLEGAL_PARAM	非法参数
0xA0028012	ES_ERR_SYS_BUSY	资源忙
0xA0028008	ES_ERR_SYS_NOT_SUPPORT	不支持的操作